3D WEBSOFT INC. FOUNDATION

SCHOOL FUTURE TECH PROGRAM

Faculty Name :

Course Duration : 5 Months (150 Days)

Classes : Offline & Online

Daily Teaching : 3 Hours

Total Hours : 300 Hours

LEVEL 3 –Future Engineers(9th–10th Standard)

MODULE 1 – Python Foundations (Classes 1–10)

Class 10 – PYTHON BASICS ACTIVITY & QUIZ

Complete Theory & Practical Notes

CLASS 1: Introduction to Python & First Python Program

Learning Objectives

By the end of the class, students will be able to:

- Understand what Python is and where it is used.
- Understand the basic structure of a Python program.
- Write and execute their first Python program.
- Use the print() function.
- Identify basic Python syntax.
- Create a simple personal-information program.

Session

Topic	Method
Introduction to Programming	Discussion
What is Python?	Theory + Visual
Python Applications	Examples
First Python Program	Demonstration
`print()` Function	Theory + Practical
Break / Coding Challenge	Activity
Strings and Numbers	Demonstration
Student Practical	Hands-on Practical
Debugging Activity	Practice
Quiz	Assessment

1. What is Python?

Python is a high-level programming language known for its readable and beginner-friendly syntax.

Python is used in:

- Web development
- Data analysis
- Artificial intelligence
- Machine learning
- Automation
- Game development
- Scientific computing

2. Your First Python Program

```
print("Hello, World!")
```

Explain:

- `print()` tells Python to display something.
- `"Hello, World!"` is the text being displayed.
- Parentheses `()` are used to provide information to the function.

3. Practical Activity – About Me

Students create:

```
print("My name is Rahul")
print("I am 15 years old")
print("I love Python")
print("I want to become a programmer")
```

Learning Outcomes

Students can:

- Explain what Python is.
- Identify common Python applications.
- Write a simple Python program.
- Use print().
- Run a basic Python program.
- Create a simple About Me program.

CLASS 2: Python Syntax, Keywords & Comments

Learning Objectives

Students will:

- Understand Python syntax.
- Identify Python keywords.
- Understand indentation and formatting.
- Create comments.
- Recognize common syntax errors.
- Practice writing clean Python code.

Session

Topic	Method
Recap of Class 1	Discussion
What is Syntax?	Theory
Python Rules	Examples

Keywords	Demonstration
Comments	Practical
Break / Error Hunt	Activity
Indentation	Demonstration
Syntax Error Practice	Hands-on
Code Cleanup Activity	Practice
Quiz	Assessment

1. What is Syntax?

Syntax means the rules used to write valid Python code.

Example:

```
print("Hello")
```

Incorrect:

```
print("Hello"
```

The second example is missing a closing parenthesis.

2. Python Keywords

Keywords are reserved words with special meanings in Python.

Examples include:

```
if
else
for
while
def
return
True
False
None
```

3. Comments

Comments help programmers explain code.

```
# This program displays a greeting
print("Hello")
```

Practical Activity – Find the Error

Give students incorrect programs and ask them to identify and correct the error

Learning Outcomes

Students can:

- Explain Python syntax.
- Recognize common syntax errors.
- Identify basic Python keywords.
- Write comments.
- Understand the importance of clean code formatting.

CLASS 3: Variables and Constants

Learning Objectives

Students will:

- Understand variables.
- Create and assign variables.
- Change variable values.
- Follow Python naming rules.
- Understand the concept of constants.
- Use variables in practical programs.

Session

Topic	Method
Recap	Discussion
What is a Variable?	Theory
Assignment	Demonstration
Variable Naming Rules	Examples

Constants	Theory
Variable Challenge	Activity
Practical Program	Hands-on
Variable Modification	Practice
Debugging	Activity
Quiz	Assessment

1. Understanding Variables

A variable is a name used to store a value.

```
name = "Anita"
age = 16
marks = 85
```

Here:

- `name` stores text.
- `age` stores a whole number.
- `marks` stores a whole number.

2. Changing Variables

```
score = 50
print(score)

score = 80
print(score)
```

The value stored in score changes.

3. Naming Rules

Good:

```
student_name = "Rahul"
student_age = 15
```

Avoid:

```
student name = "Rahul"
```

Practical – Student Profile

```
name = "Rahul"
age = 15
school = "ABC School"
city = "Hyderabad"

print(name)
print(age)
print(school)
print(city)
```

Learning Outcomes

Students can:

- Define a variable.
- Create variables.
- Assign values to variables.
- Follow basic naming rules.
- Modify variable values.
- Create a simple student-profile program.

CLASS 4: Data Types

Learning Objectives

Students will:

- Understand why data has different types.
- Identify `str`, `int`, `float`, and `bool`.
- Use `type()`.
- Store different types of data in variables.
- Connect Python data types to real-world information.

Session

Topic	*Method*
Recap	Discussion
What is Data?	Theory
Strings	Theory + Practical
Integers	Examples
Floats	Examples

Data Type Sorting	Activity
Break / Challenge	Activity
Boolean	Theory
`type()`	Practical
Student Profile	Hands-on
Quiz	Assessment

Main Data Types

String

```
name = "Rahul"
```

Type:

```
str
```

Integer

```
age = 15
```

Type:

```
int
```

Float

```
height = 160.5
```

Type:

```
float
```

Boolean

```
is_student = True
```

Type:

```
bool
```

Checking Data Types

```
name = "Rahul"
age = 15
height = 160.5
is_student = True

print(type(name))
print(type(age))
print(type(height))
```

```
print(type(is_student))
```

Practical – Data Type Sorting Game

Give students examples:

```
"Python"
25
75.5
True
"Student"
100
```

Students classify each as:

- String
- Integer
- Float
- Boolean

Learning Outcomes

Students can:

- Explain basic Python data types.
- Identify strings, integers, floats, and booleans.
- Use type().
- Store different types of information.
- Choose an appropriate data type for simple problems.

CLASS 5: Input & Output

Learning Objectives

Students will:

- Understand input and output.
- Use print() for output.
- Use input() for user input.
- Store input in variables.
- Understand that input() returns a string.
- Convert input into numbers using int() and float().

Session

Topic	Method
Communication Warm-up	Activity
Input vs Output	Theory
`print()`	Practical
Printing Variables	Demonstration
`input()`	Theory + Practical
Personal Information Program	Hands-on
Break / Input Challenge	Activity
Input as String	Demonstration
`int()` and `float()`	Practical
Student Details Program	Hands-on
Quiz	Assessment

Input and Output

Input = information given to a program.

Output = information produced by a program.

Basic flow:

```
Input → Processing → Output
```

Output

```
print("Welcome to Python")
```

Input

```
name = input("What is your name? ")
print("Hello", name)
```

Numeric Input

```
age = int(input("Enter your age: "))
print("Your age is", age)
```

Float:

```
height = float(input("Enter your height: "))
print("Height:", height)
```

Practical – Personal Information

```
name = input("Enter your name: ")
age = input("Enter your age: ")
city = input("Enter your city: ")

print("===== MY PROFILE =====")
print("Name:", name)
print("Age:", age)
print("City:", city)
```

Challenge – Simple Calculator

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Sum:", num1 + num2)
```

Learning Outcomes

Students can:

- Explain input and output.
- Use print().
- Use input().
- Store user input.
- Convert text input into numbers.
- Create an interactive Python program.

CLASS 6: Operators in Python

Learning Objectives

Students will:

- Understand operators and operands.
- Use arithmetic operators.
- Perform calculations in Python.
- Understand division, floor division, and remainder.
- Use operators in real-world programs.
- Understand basic operator precedence.

Session

Topic ***Method***

Math Warm-up	Activity
Operators & Operands	Theory
Arithmetic Operators	Demonstration
/, //, %	Practical
Power **	Examples
Calculator Program	Hands-on
Break / Operator Challenge	Activity
Expressions & Precedence	Theory
Shopping Bill	Practical
Marks Calculator	Hands-on
Quiz	Assessment

Arithmetic Operators

Operator	Meaning	Example
+	Addition	`10 + 5`
-	Subtraction	`10 - 5`
*	Multiplication	`10 * 5`
/	Division	`10 /5`
%	Remainder	`10 % 3`
//	Floor Division	`10 // 3`
**	Power	`2 ** 3`

Calculator Practical

```
num1 = 20
```

```
num2 = 5

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
print("Floor Division:", num1 // num2)
print("Remainder:", num1 % num2)
print("Power:", num1 ** 2)
```

Operator Precedence

Python generally evaluates:

1. Parentheses ()
2. Power **
3. Multiplication/division/floor division/modulus
4. Addition/subtraction

Example:

```
result = 10 + 5 * 2
print(result)
```

Output:

```
20
```

Using parentheses:

```
result = (10 + 5) * 2
print(result)
```

Output:

```
30
```

Practical – Shopping Bill

```
item_price = float(input("Enter item price: "))
quantity = int(input("Enter quantity: "))

total = item_price * quantity

print("Total Bill:", total)
```

Learning Outcomes

Students can:

- Explain operators and operands.
- Use arithmetic operators.
- Create mathematical expressions.
- Use %, //, and **.
- Apply operator precedence.
- Build simple calculation programs.

CLASS 7: Comparison Operators

Learning Objectives

Students will:

- Understand comparison operators.
- Compare two values.
- Understand Boolean results.
- Use ==, !=, >, <, >=, and <=.
- Apply comparisons to real-world situations.
- Prepare for decision-making using if.

Session

Topic	Method
Comparison Warm-up	Activity
What is Comparison?	Theory
Comparison Operators	Demonstration
Boolean Results	Practical
Real-world Examples	Discussion
Break / Comparison Game	Activity
User Input Comparisons	Hands-on
Age Eligibility Activity	Practical
Debugging	Practice
Quiz	Assessment

Comparison Operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
age = 18

print(age == 18)
print(age > 15)
print(age < 10)
```

The result of a comparison is normally:

```
True
Fals
```

Important Difference

```
x = 10
```

means assignment.

```
x == 10
```

means comparison.

Practical – Age Checker

```
age = int(input("Enter your age: "))

print("Age is 18 or above:", age >= 18)
print("Age is below 18:", age < 18)
```

Activity – Comparison Cards

Teacher gives pairs:

```
10 and 5
20 and 20
```

```
7 and 15
100 and 50
```

Students predict whether each comparison is True or False.

Learning Outcomes

Students can:

- Identify comparison operators.
- Compare values.
- Predict Boolean results.
- Distinguish = from ==.
- Create simple comparison programs.

CLASS 8: Logical Operators

Learning Objectives

Students will:

- Understand logical operators.
- Use and, or, and not.
- Combine multiple conditions.
- Understand basic logical reasoning.
- Apply logical operators to real-world situations.

Session

Topic	*Method*
Logic Warm-up	Activity
Introduction to Logic	Theory
`and`	Demonstration
`or`	Demonstration
`not`	Demonstration
Logic Game	Activity
Break	—

Combining Conditions	Practical
Login/Eligibility Example	Hands-on
Challenge	Practice
Quiz	Assessment

and

Both conditions must be true.

```
age = 20
has_id = True

print(age >= 18 and has_id)
```

or

At least one condition must be true.

```
has_ticket = True
is_guest = False

print(has_ticket or is_guest)
```

not

Reverses a Boolean value.

```
is_raining = False

print(not is_raining)
```

Practical – Eligibility

```
age = int(input("Enter your age: "))
has_permission = input("Do you have permission? ")

print(age >= 18 and has_permission == "yes")
```

Activity – Logic Cards

Students solve statements such as:

```
Age >= 18 AND has ID
Has ticket OR is invited
NOT is_raining
```

Suitable Visual

Learning Outcomes

Students can:

- Explain logical operators.
- Use `and`, `or`, and `not`.
- Combine conditions.
- Predict basic Boolean results.
- Build simple logic-based programs.

CLASS 9: Type Conversion & Practical Problem Solving

Learning Objectives

Students will:

- Understand type conversion.
- Convert strings into integers and floats.
- Convert numbers into strings.
- Understand why conversion is needed.
- Combine input, variables, data types, and operators.
- Build a small practical application.

2-Hour Schedule

Time	Topic	Method
0–10 min	Data Type Recap	Discussion
10–25 min	What is Type Conversion?	Theory
25–40 min	`int()`	Demonstration
40–52 min	`float()`	Demonstration
52–64 min	`str()`	Demonstration
64–72 min	Conversion Challenge	Activity
72–80 min	Break	—
80–95 min	Input + Conversion	Practical
95–108 min	Shopping Calculator	Hands-on
108–116 min	Mini Project	Practice
116–120 min	Quiz	Assessment

Why Type Conversion?

Remember:

```
age = input("Enter age: ")
```

The value from `input()` is normally a string.

If we want to perform calculations, convert it:

```
age = int(input("Enter age: "))
```

Integer Conversion

```
number = int("25")
print(number + 5)
```

Float Conversion

```
price = float("99.50")
print(price)
```

String Conversion

```
age = 15
message = "I am " + str(age) + " years old."

print(message)
```

Practical – Shopping Calculator

```
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))

total = price * quantity

print("Product Price:", price)
print("Quantity:", quantity)
print("Total:", total)
```

Mini Project – Student Marks

```
maths = float(input("Enter Maths marks: "))
science = float(input("Enter Science marks: "))
english = float(input("Enter English marks: "))

total = maths + science + english
average = total / 3

print("===== MARKS REPORT =====")
print("Total:", total)
print("Average:", average)
```

Suitable Visual

Learning Outcomes

Students can:

- Explain type conversion.
- Use `int()`, `float()`, and `str()`.
- Convert user input into useful data.
- Perform calculations using converted values.
- Combine multiple Python concepts in one program.

CLASS 10: Python Basics Activity & Quiz

Learning Objectives

Students will:

- Review the main concepts from Classes 1–9.
- Apply Python fundamentals independently.
- Combine variables, data types, input/output, and operators.
- Solve beginner-level programming problems.
- Identify and correct basic errors.
- Demonstrate understanding through activities and a quiz.

2-Hour Schedule

Time	Topic	Method
0–15 min	Module Revision	Discussion
15–30 min	Python Concept Review	Theory
30–45 min	Code Reading Activity	Practical
45–60 min	Debugging Challenge	Hands-on
60–70 min	Break	—
70–90 min	Python Basics Activity	Group/Individual
90–105 min	Mini Programming Challenge	Practical
105–115 min	Quiz	Assessment
115–120 min	Review & Feedback	Discussion

Revision Topics

Students revise:

1. Python basics
2. `print()`
3. Syntax
4. Comments
5. Variables
6. Constants
7. Data types
8. `input()`
9. Type conversion
10. Arithmetic operators
11. Comparison operators
12. Logical operators

Activity 1 – Predict the Output

Give students:

```
name = "Ravi"
age = 15

print("Name:", name)
print("Age:", age)
```

Ask:

What will appear on the screen?

Students write their answers before running the program.

Activity 2 – Find the Error

```
name = input("Enter your name: "
print("Hello", name)
```

Students identify the missing syntax element and correct the program.

Correct version:

```
name = input("Enter your name: ")
print("Hello", name)
```

Activity 3 – Build a Student Profile

Students create a program containing:

- Name
- Age
- School
- City
- Favorite subject

Example:

```
name = input("Name: ")
age = int(input("Age: "))
school = input("School: ")
city = input("City: ")
subject = input("Favorite subject: ")

print()
print("===== STUDENT PROFILE =====")
print("Name:", name)
print("Age:", age)
print("School:", school)
print("City:", city)
print("Favorite Subject:", subject)
```

Activity 4 – Simple Bill Calculator

```
price = float(input("Enter price: "))
quantity = int(input("Enter quantity: "))

total = price * quantity

print("Total Bill:", total)
```

Challenge Questions

Ask students to extend the program by adding:

- Discount
- Tax
- Final bill
- Customer name

Activity 5 – Marks Calculator

Students build:

```
maths = float(input("Maths: "))
science = float(input("Science: "))
english = float(input("English: "))

total = maths + science + english
```

```
average = total / 3

print("Total:", total)
print("Average:", average)
```

Extension Challenge

Ask students to add:

```
Student Name
Total Marks
Average
```

and display them as a report.

Python Basics Quiz

Use the interactive quiz below for the end-of-module assessment.

Suitable Visuals for Revision

Teacher Preparation for Classes 1–10

Before each class, the teacher should prepare:

- Python installed on student computers, or a browser-based Python environment.
- Projector/display for live coding.
- Prepared `.py` starter files.
- Worksheets for activities.
- Beginner-friendly coding exercises.
- Quiz questions.
- Debugging examples.
- Whiteboard/marker.
- Student practice checklist.

Recommended Teaching Pattern

Each 2-hour session should follow this pattern:

Explain → Demonstrate → Practice Together → Independent Practice → Challenge → Review → Quiz

The teacher should avoid spending the entire class on theory. Students should write and run code during every session.

MODULE 1 – FINAL LEARNING OUTCOMES

After completing Classes 1–10, students will be able to:

1. Explain what Python is.
2. Write and run a basic Python program.
3. Use `print()` for output.
4. Use `input()` for user interaction.
5. Understand basic Python syntax.
6. Write and use comments.
7. Create and modify variables.
8. Understand basic constants.
9. Identify common Python data types.
10. Use `type()`.
11. Convert between common data types.
12. Use arithmetic operators.
13. Use comparison operators.
14. Use logical operators.
15. Build simple interactive programs.
16. Perform basic calculations.
17. Read and understand simple Python code.
18. Identify and correct beginner-level errors.
19. Solve small programming problems independently.
20. Demonstrate foundational Python skills through activities and quizzes.